

UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

SmartHome IoT: Sistema de Automatización, Monitoreo y Control de Acceso para una Casa Inteligente

Informe Técnico - Proyecto Unidad 1 (Equipo 69)

Integrantes:

Mariano Muñoz R.

Cristian Ruiz I.

Camilo Fuentes G.

Profesor:

Ricardo Pérez

9 de Junio de 2026

Índice

1. Introducción y Descripción del Problema	2
1.1. Introducción	2
1.2. Descripción del Problema	2
2. Objetivos del Proyecto	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	3
3. Arquitectura del Sistema	4
3.1. Arquitectura General de Componentes	4
3.2. Secuencia de Control de Acceso Biométrico	5
3.3. Secuencia de Monitoreo y Reglas de Control	6
4. Inventario e Integración de Sensores y Actuadores	7
4.1. Descripción de los Sensores y su Rol	7
4.2. Actuadores	8
4.3. Inventario Físico de Componentes	8
4.4. Conexiones Físicas y Asignación de Pines	9
5. Comunicación MQTT y Estructura de Tópicos	9
5.1. Jerarquía de Tópicos Requerida	9
5.2. Formatos de Mensaje JSON	10
6. Estrategias de Control Automático y Notificaciones	11
6.1. Reglas de Automatización de Emergencias	11
6.2. Prevención de Alertas Repetitivas (Cooldown) y Enclavamiento	12
7. Dashboard y Canales de Interfaz	12
7.1. Panel de Monitoreo Node-RED Dashboard	12
7.2. Interfaz Web de Acceso Personalizada	12
7.3. Bot de Telegram	13
7.4. Registro Histórico de Datos	13
8. Problemas Encontrados y Soluciones Aplicadas	13
8.1. Problema 1: Apagado Inexistente del LED de Alerta	14
8.2. Problema 2: Inestabilidad y Pérdida en el Reconocimiento Facial	14
8.3. Problema 3: Caídas Frecuentes a Cero del Sensor de Gas MQ-2	15
9. Conclusiones	15

1. Introducción y Descripción del Problema

1.1. Introducción

El desarrollo del Internet de las Cosas (IoT) ha revolucionado la forma en que interactuamos con nuestros entornos cotidianos. La domótica, o automatización del hogar, ya no es un concepto futurista exclusivo, sino una realidad accesible que permite integrar sensores, actuadores y algoritmos de control para mejorar significativamente tres pilares fundamentales de la vida moderna: la seguridad física de los habitantes, el confort térmico y ambiental del espacio habitable, y la eficiencia energética mediante el control inteligente de recursos.

En este proyecto, se diseña e implementa un prototipo completo y funcional de casa inteligente. A diferencia de las soluciones comerciales cerradas, este sistema se fundamenta en un diseño distribuido y modular que combina hardware de desarrollo real (Arduino MKR1000 y ESP32-CAM), protocolos de comunicación estándar en la industria (MQTT), una plataforma visual para lógica y paneles de control local (Node-RED), y una capa intermedia de visión artificial con base de datos dedicada (FastAPI + SQLite) para la autenticación biométrica de accesos.

1.2. Descripción del Problema

El caso de estudio involucra a una familia que desea automatizar su residencia debido a diversas necesidades cotidianas insatisfechas. En primer lugar, la seguridad del hogar es una preocupación crítica: requieren un control de acceso automatizado en la puerta principal que permita identificar visualmente si hay una persona presente y autorizar la entrada únicamente si esta pertenece al núcleo familiar, registrando cada evento de acceso e identificando rostros desconocidos para su posterior enrolamiento. En segundo lugar, existe la necesidad de monitorear en tiempo real variables ambientales para asegurar el confort y la salud, específicamente temperatura, humedad e indicadores acústicos. Por último, se requiere un esquema de alerta temprana y automatizada ante anomalías críticas del entorno, tales como fugas de gas licuado o humos nocivos, que actúe de manera local activando alarmas visuales y sonoras, y de forma externa enviando notificaciones detalladas a los celulares de la familia para permitirles reaccionar oportunamente incluso si se encuentran fuera del hogar.

El prototipo desarrollado resuelve estas problemáticas mediante un diseño distribuido que integra hardware real de sensores y actuadores con contenedores virtualizados que simulan una infraestructura en la nube o servidor local dentro del hogar.

2. Objetivos del Proyecto

2.1. Objetivo General

Diseñar, implementar y evaluar un prototipo funcional de sistema IoT para la automatización, monitoreo ambiental y control de acceso biométrico de una vivienda inteligente, utilizando sensores físicos y actuadores conectados, comunicación por mensajería

MQTT, procesamiento de imágenes por visión artificial y reglas de control en un panel visual integrado.

2.2. Objetivos Específicos

1. Programar y conectar un nodo sensor/actuador basado en Arduino MKR1000 para leer de manera robusta variables físicas del entorno y controlar actuadores locales de alerta y acceso.
2. Integrar una cámara ESP32-CAM al broker MQTT para transmitir ráfagas de imágenes JPEG a demanda ante eventos específicos del sistema.
3. Establecer la comunicación de red inalámbrica mediante un broker MQTT Mosquitto centralizado, utilizando una jerarquía de tópicos consistente y estructurando la telemetría en formato JSON.
4. Diseñar e implementar un flujo en Node-RED que actúe como panel de control principal (Dashboard) para visualizar datos históricos y en tiempo real del sistema.
5. Desarrollar e integrar un backend en Python (FastAPI) con la librería `face_recognition` basada en dlib para el procesamiento biométrico de rostros, controlando la apertura automática de la puerta y el enrolamiento temporal de desconocidos.
6. Codificar estrategias de automatización basadas en umbrales de seguridad para alertas de temperatura, gas y sonido en Node-RED.
7. Integrar notificaciones externas de mensajería (Telegram Bot) para el reporte instantáneo de emergencias y la recepción de comandos remotos.
8. Registrar históricamente la actividad del sistema mediante un archivo plano CSV local y una base de datos relacional SQLite que garanticen la persistencia de datos.

3. Arquitectura del Sistema

El sistema se diseña bajo un esquema de arquitectura distribuida y desacoplada, donde cada componente se comunica de forma asíncrona mediante el broker de mensajería MQTT. A continuación se detallan los aspectos lógicos y físicos de esta solución.

3.1. Arquitectura General de Componentes

La topología del sistema consta de cinco bloques principales coordinados a través de la red local:

1. **Nodo Sensor (Arduino MKR1000)**: Lee periódicamente las variables de temperatura, humedad, gas analógico, gas digital y amplitud de sonido. Actúa de forma pasiva a la espera de comandos MQTT para activar el LED de alerta, el buzzer o el LED indicador de puerta.
2. **Cámara IoT (ESP32-CAM)**: Se conecta por WiFi directamente al broker MQTT. Permanece en estado de reposo hasta recibir un comando en el tópico de captura, momento en el cual realiza una ráfaga rápida de capturas a alta velocidad, enviando los bytes JPEG directamente por MQTT.
3. **Broker MQTT (Mosquitto)**: Contenedor Docker que actúa como el bus de datos central. Redirige los mensajes de telemetría y comandos con configuraciones de calidad de servicio (QoS) apropiadas.
4. **Motor de Reglas y Dashboard (Node-RED)**: Contenedor Docker encargado de recibir la telemetría de sensores, actualizar el panel web de Node-RED Dashboard, evaluar umbrales críticos de seguridad, enviar notificaciones externas por Telegram y registrar las lecturas en un archivo CSV local montado en el host.
5. **Servicio de Acceso y Visión (FastAPI Backend y Nginx Frontend)**: El backend en FastAPI procesa las imágenes de la cámara, realiza la comparación geométrica de rostros con encodings almacenados en una base de datos SQLite y publica comandos de apertura de puerta. El frontend (servido por Nginx) es un panel web alternativo especializado en el monitoreo del acceso de personas y el enrolamiento en caliente.

Para visualizar la interconexión física y lógica de los componentes, consulte la Figura [1](#).

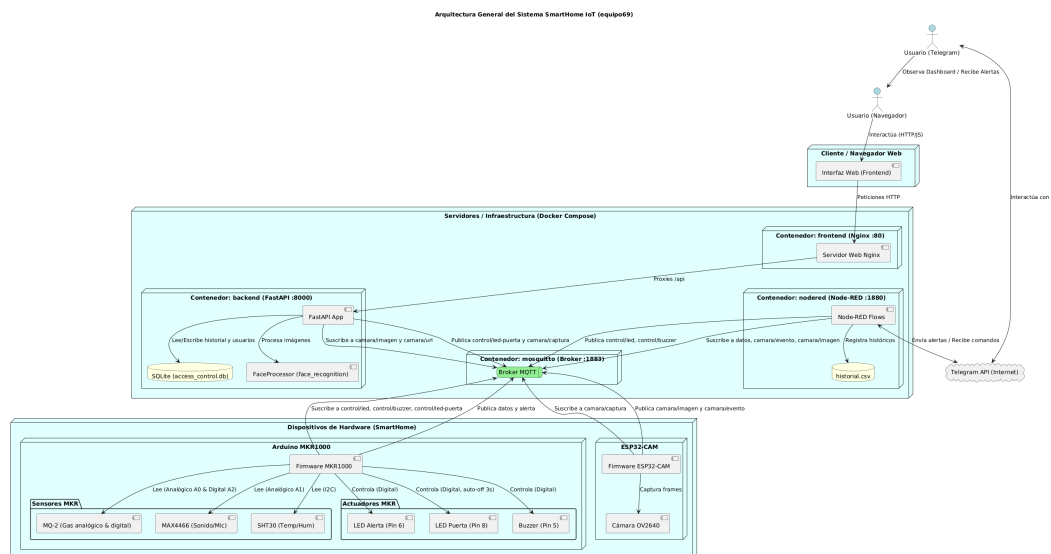


Figura 1: Arquitectura General del Sistema y Conectividad de Red (arqui_1). Al hacer clic en la imagen se abrirá el SVG de PlantUML en alta resolución.

3.2. Secuencia de Control de Acceso Biométrico

El control de acceso por reconocimiento facial opera mediante una secuencia asíncrona optimizada para evitar la sobrecarga de la red local y proteger el hardware de la cámara. El flujo principal se detalla en la Figura 2 y consta de las siguientes etapas clave:

- **Activación:** El usuario solicita una apertura desde el panel frontend o el sensor de sonido detecta un ruido inusual. El backend FastAPI publica una orden de ráfaga (`iniciar_burst`) hacia la cámara.
- **Captura y Transmisión:** La ESP32-CAM captura imágenes VGA de forma no bloqueante a una tasa fija de 4 frames por segundo durante 5 segundos (20 frames en total) y los transmite como bytes crudos de datos binarios mediante MQTT.
- **Procesamiento e Identificación:** El backend FastAPI recibe estos frames y los procesa en sub-ráfagas de 3 segundos mediante hilos de ejecución paralelos con `face_recognition`. Si se encuentra una cara conocida con una confianza mínima de 50 % y una tolerancia estricta de 0.35 en al menos 3 frames consecutivos, se concede el acceso.
- **Apertura de Puerta:** Al autorizar el acceso, se guarda el evento en la base de datos SQLite como “Entrada Automática” y se publica la orden de encendido de la puerta por MQTT. El Arduino MKR1000 recibe esta orden, enciende el LED físico de la puerta y ejecuta un temporizador local no bloqueante de 3 segundos para apagarlo automáticamente.
- **Enrolamiento:** Si no se reconoce el rostro, el backend guarda el evento “Desconocido Detectado”, extrae y almacena temporalmente el frame en caché, y abre

una ventana de enrolamiento de 60 segundos. El frontend detecta esta condición mediante polling al endpoint de eventos, activa un temporizador visual y permite al usuario registrar al visitante escribiendo su nombre, guardando su encoding en la base de datos de manera definitiva.

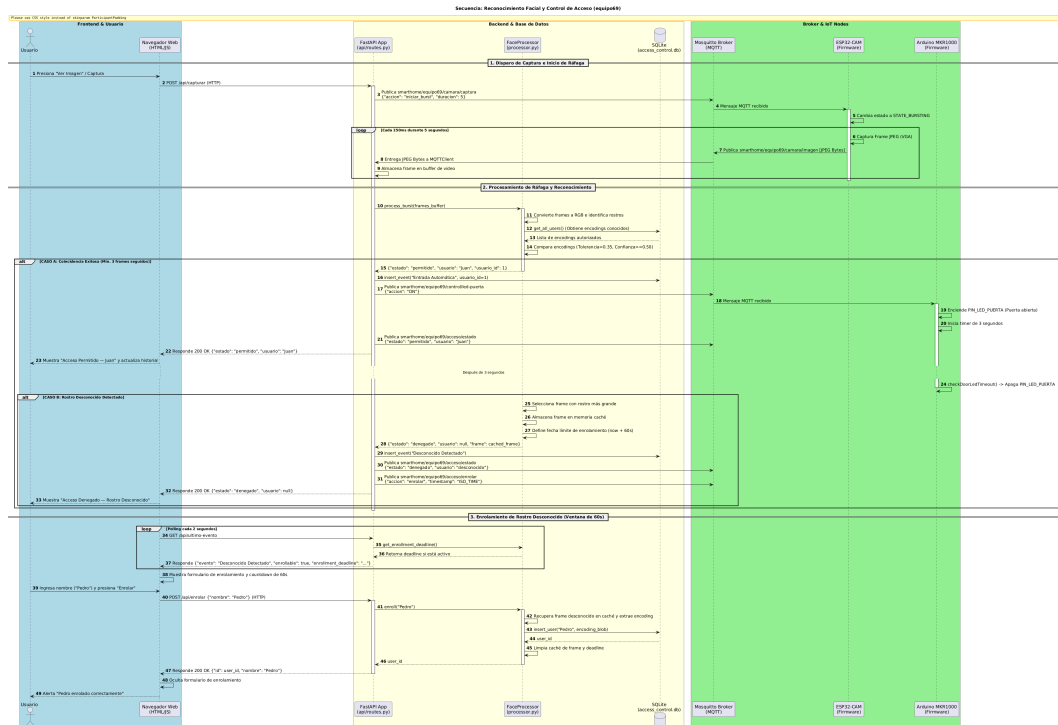


Figura 2: Diagrama de Secuencia de Control de Acceso y Enrolamiento Facial (arqui_2). Al hacer clic en la imagen se abrirá el SVG de PlantUML en alta resolución.

3.3. Secuencia de Monitoreo y Reglas de Control

El monitoreo ambiental y las reglas de seguridad automáticas se ejecutan de forma paralela e independiente en Node-RED. La Figura 3 describe el flujo desde que los sensores físicos leen las variables ambientales hasta la activación de alertas y notificaciones.



Figura 3: Diagrama de Secuencia de Monitoreo de Telemetría y Alertas Automáticas (arqui_3). Al hacer clic en la imagen se abrirá el SVG de PlantUML en alta resolución.

4. Inventario e Integración de Sensores y Actuadores

4.1. Descripción de los Sensores y su Rol

Para lograr un monitoreo completo del entorno y garantizar el correcto funcionamiento del control de accesos, se han integrado los siguientes dispositivos:

1. **Sensor de Temperatura y Humedad SHT30 (I2C):** Conectado a los pines de hardware dedicados I2C (SDA Pin 11, SCL Pin 12) del Arduino MKR1000. Su rol es proveer mediciones precisas del estado térmico y humedad del aire, variables esenciales para el confort y la detección de incendios o sobrecalentamiento de espacios.
2. **Sensor de Gas Licuado y Humo MQ-2 (Analógico/Digital):** Dispositivo electroquímico que mide la concentración de gases combustibles en el aire. Conectado al pin analógico A0 para monitorear el nivel absoluto de gas y al pin digital

A2 como entrada de disparo rápida basada en el comparador del propio sensor físico. Su rol es emitir alertas críticas ante fugas o presencia de fuego.

3. **Micrófono y Amplificador MAX4466 (Sonido - Analógico):** Conectado al pin analógico A1. Mide las ondas de presión acústica realizando un muestreo continuo de pico a pico durante 50 milisegundos para determinar la amplitud del sonido ambiente. Actúa como sensor diferencial complementario, configurado como disparador acústico del sistema de seguridad.
4. **Módulo de Cámara ESP32-CAM:** Tarjeta con microcontrolador ESP32-S y sensor OV2640. Su rol exclusivo es capturar y codificar imágenes JPEG bajo demanda tras recibir órdenes del bus MQTT, permitiendo al sistema de visión artificial en FastAPI realizar reconocimiento de personas en el hogar.

4.2. Actuadores

El sistema cuenta con actuadores para señalización y respuesta física inmediata:

- **LED de Alerta General (Pin 6):** LED amarillo que se activa de forma continua al detectarse temperaturas críticas o niveles altos de gas licuado.
- **Buzzer de Alerta General (Pin 5):** Generador de tonos sonoros para llamar la atención en la vivienda ante alarmas de seguridad activas.
- **LED Indicador de Puerta (Pin 8):** LED verde que simula la cerradura eléctrica de acceso. Se enciende por 3 segundos tras verificarse la identidad de un habitante autorizado.

4.3. Inventario Físico de Componentes

El Cuadro 1 y el Cuadro 2 detallan el stock físico utilizado en el prototipo de laboratorio.

Tabla 1: Inventario de Placas de Desarrollo y Sensores

Componente	Modelo/Descripción	Voltaje de Trabajo	Cantidad
Placa Principal	Arduino MKR1000 WiFi	3.3 V (Micro USB)	1
Cámara IoT	ESP32-CAM (con Shield MB)	5.0 V (USB)	1
Sensor Ambiental	SHT30 (Temperatura y Humedad)	3.3 V	1
Sensor de Gas	MQ-2 Gas/Humo	5.0 V	1
Micrófono/Sonido	MAX4466 con amplificador ajustable	3.3 V	1

Tabla 2: Inventario de Componentes Pasivos y Accesorios

Componente	Color / Detalle	Cantidad
Potenciómetros	Ajuste de ganancia y umbral	4
LED de Alerta	Amarillo (Señalización de peligro)	7
LED de Entrada	Verde (Simulador de electro-chapa)	9
LED de Bloqueo	Rojo (Acceso Denegado)	10
LED de Red	Azul (Estado de conexión WiFi)	4

4.4. Conexiones Físicas y Asignación de Pines

La asignación de pines configurados en el firmware del Arduino MKR1000 (centralizados en `src/config.h`) se presenta en el Cuadro 3.

Tabla 3: Asignación de Pines Físicos en el Arduino MKR1000

Componente	Pin MKR1000	Dirección de Señal	Tipo de Señal
LED Alerta General	Pin 6	Salida	Digital (PWM)
Buzzer de Alerta	Pin 5	Salida	Digital (PWM)
LED Cerradura Puerta	Pin 8	Salida	Digital (PWM)
MQ-2 Analógico (AO)	Pin A0	Entrada	Analógica (ADC)
MQ-2 Digital (DO)	Pin A2	Entrada	Digital
MAX4466 Sonido (OUT)	Pin A1	Entrada	Analógica (ADC)
SHT30 Datos (SDA)	Pin 11	Bidireccional	I2C Hardware Fijo
SHT30 Reloj (SCL)	Pin 12	Salida	I2C Hardware Fijo

Las conexiones del sensor SHT30 se identifican mediante un cableado de cuatro hilos estructurado por colores normalizados: cable rojo para VCC (3.3V), cable negro para GND, cable verde para la línea SDA (Pin 11) y cable amarillo para la línea SCL (Pin 12).

5. Comunicación MQTT y Estructura de Tópicos

La mensajería MQTT (Message Queuing Telemetry Transport) es el núcleo de comunicación del sistema domótico. Se utiliza un broker Mosquitto configurado de forma aislada en Docker Compose accesible localmente a través del puerto estándar 1883.

5.1. Jerarquía de Tópicos Requerida

Todos los tópicos del proyecto se estructuran bajo una jerarquía jerárquica limpia que utiliza el identificador del equipo asignado (`equipo69`) como namespace central.

Esto garantiza la separación de mensajes en el caso de compartir un broker común. La estructura detallada se resume en el Cuadro 4.

Tabla 4: Estructura de Tópicos MQTT del Sistema (Namespace: smarthome/equipo69/)

Tópico Relativo	Dirección	QoS	Retain	Propósito
datos	MKR → Broker	0	No	Envío de JSON con telemetría general
alerta	MKR → Broker	1	No	Texto plano indicando alarma crítica
control/led	Broker → MKR	1	Sí	Comando “ON”/“OFF” para LED de alerta
control/buzzer	Broker → MKR	1	Sí	Comando “ON”/“OFF” para el Buzzer
control/led-puerta	Broker → MKR	1	Sí	JSON para encender/apagar la cerradura
camara/evento	ESP → Broker	1	No	JSON indicando estado de la cámara
camara/captura	API → ESP	1	No	JSON ordenando ráfaga de fotos
camara/imagen	ESP → Broker	1	No	Datos binarios de imágenes (JPEG)
acceso/estado	API → Broker	1	No	JSON con veredicto de reconocimiento
acceso/enrolar	API → Broker	1	No	JSON notificando ventana de enrolamiento

5.2. Formatos de Mensaje JSON

Para simplificar el procesamiento en Node-RED y el backend FastAPI, los datos de sensores del Arduino MKR1000 se empaquetan en un único documento JSON estructurado que se publica de forma asíncrona cada 2 segundos. Si algún sensor físico experimenta una falla o desconexión, la clave asociada toma un valor nulo de manera explícita (en lugar de enviar 0,0, cumpliendo con las especificaciones técnicas).

El listado 1 muestra un ejemplo real del mensaje JSON publicado en el tópico smarthome/equipo69/datos bajo condiciones de operación normal.

```

1 {
2   "equipo": "equipo69",
3   "temperatura": 23.5,
4   "humedad": 61.2,
5   "gas": 412,
6   "gas_digital": "NORMAL",
7   "sensor_extra": 128
8 }
```

Listing 1: Ejemplo de payload JSON publicado por el nodo de sensores.

En caso de que el sensor ambiental SHT30 falle o esté desconectado eléctricamente, el JSON se reestructura automáticamente omitiendo los valores erróneos o reemplazándolos por valores nulos, tal como se ilustra en el Listado 2.

```

1 {
2   "equipo": "equipo69",
3   "temperatura": null,
4   "humedad": null,
5   "gas": 415,
6   "gas_digital": "NORMAL",
```

```
7 "sensor_extra": 130
8 }
```

Listing 2: Ejemplo de payload JSON bajo falla del sensor SHT30.

El comando para accionar la puerta enviado al tópico `smarthome/equipo69/control/led-puerta` utiliza la estructura: `{.accion": ".ON"}` o `{.accion": ".OFF"}`. Por otro lado, la orden de ráfaga a la cámara en el tópico `smarthome/equipo69/camara/captura` utiliza: `{.accion": "iniciar_burst", "duracion": 5}`.

6. Estrategias de Control Automático y Notificaciones

El sistema implementa una lógica de control de bucle cerrado basada en reglas de decisión programadas en Node-RED y en el backend FastAPI, respondiendo de manera autónoma ante emergencias.

6.1. Reglas de Automatización de Emergencias

Las tres estrategias principales de control automático implementadas en Node-RED son:

1. **Control por Temperatura Crítica:** Si la lectura analizada de temperatura supera los 30°C, Node-RED publica comandos “ON” independientes a los tópicos de control de actuadores locales (`control/led` y `control/buzzer`), y genera un mensaje con formato Markdown enriquecido dirigido al bot de Telegram. La alarma permanece activa hasta que la lectura disminuya de los 30°C y transcurra el tiempo de enclavamiento.
2. **Control por Gas Crítico:** Cuando el valor analógico del MQ-2 supera las 1020 ppm (o la entrada digital DO pasa a nivel LOW), se asume la presencia de gases combustibles nocivos. Node-RED activa de forma inmediata el LED de alerta y el buzzer, ordena a la cámara capturar una ráfaga mediante la publicación en `camara/captura`, y despacha una alerta roja por Telegram indicando la concentración exacta detectada.
3. **Disparo de Seguridad por Ruido Fuerte (MAX4466):** El sensor de sonido de pico a pico (`sensor_extra`) actúa como centinela en la noche. Si el nivel acústico sobrepasa el umbral dinámico de 150 unidades, el sistema infiere un evento sospechoso (ej. golpe o rotura de cristal) y gatilla automáticamente una orden de ráfaga a la cámara ESP32-CAM. Las imágenes se procesan de inmediato en busca de rostros.

6.2. Prevención de Alertas Repetitivas (Cooldown) y Enclavamiento

Para evitar la saturación de los canales de comunicación externos y proteger la tranquilidad de la familia, se han integrado mecanismos de software en los flujos de Node-RED:

- **Cooldown de Notificaciones:** Cuando una regla crítica se dispara, se genera una notificación externa y se almacena la hora del envío en la memoria temporal de Node-RED (`flow.set("lastAlertTime", Date.now())`). El sistema bloquea nuevos despachos por Telegram durante los siguientes 60 segundos, aunque la variable crítica siga por encima del umbral.
- **Latching (Retención) y Temporizador de 15 Minutos:** A nivel físico, un LED oscilando rápidamente debido a variaciones de ruido de los sensores genera desgaste y fatiga. Si los sensores vuelven a rangos seguros, el LED y el buzzer no se apagan inmediatamente. Node-RED mantiene el estado "ON" por un mínimo de 15 minutos (900,000 milisegundos) desde el último valor crítico leído. Este temporizador se evalúa de manera asíncrona no bloqueante utilizando comparaciones de tiempo en milisegundos (`millis()`).

7. Dashboard y Canales de Interfaz

7.1. Panel de Monitoreo Node-RED Dashboard

La interfaz de usuario principal de Node-RED se sirve en la URL `http://localhost:1880/ui` y contiene widgets interactivos organizados en tres grupos:

- **Sensores en Tiempo Real:** Dos relojes analógicos de aguja (Gauges) para Temperatura (0–50°C) y Humedad (0–100 %), un indicador tipo brújula de ppm de Gas MQ-2 (0–1024), y un campo de texto dinámico que muestra la intensidad acústica del MAX4466.
- **Seguridad e Imagen de Cámara:** Un widget de plantilla HTML que despliega en Base64 la última fotografía transmitida por la ESP32-CAM, un indicador circular de estado (verde para estado normal, rojo para alerta de intrusión o emergencia) y texto descriptivo de rostros detectados.
- **Controles Manuales e Historial:** Botones virtuales para encender y silenciar manualmente el LED y el buzzer de alerta, y un gráfico de líneas dinámico que despliega la curva histórica de temperatura de las últimas 5 horas.

7.2. Interfaz Web de Acceso Personalizada

El sistema cuenta con un segundo frontend especializado en control de accesos desarrollado en vanilla HTML5/JS y servido por Nginx en el puerto 80. Este panel consulta la API REST de FastAPI mediante polling cada 2 segundos. Muestra la cámara en

vivo relay a través de un stream MJPEG dinámico, visualiza una tabla ordenada con el historial de accesos reciente extraído de la base de datos y despliega el formulario temporal de enrolamiento para registrar rostros detectados como desconocidos.

7.3. Bot de Telegram

El bot de Telegram (`SmartHomeBot`) implementa comandos bidireccionales administrados por Node-RED:

- `/status`: Responde inmediatamente con un mensaje estructurado detallando las lecturas más recientes almacenadas en el contexto global del sistema (Temperatura, Humedad, Gas y Sonido).
- `/ayuda` o `/start`: Despliega el menú interactivo con la descripción de los comandos soportados.
- `/entendido` o `/noted`: Comando prioritario de confirmación de alerta. Permite al propietario apagar de forma remota y segura los actuadores de emergencia (LED de Alerta y Buzzer) antes de que venza el enclavamiento de 15 minutos en Node-RED.

7.4. Registro Histórico de Datos

La persistencia de la telemetría e incidentes del sistema se asegura en dos niveles:

1. **Historial Ambiental (CSV)**: Node-RED concatena cada lectura en una cadena separada por comas con formato: `timestamp,temperatura,humedad,gas,sensor_extra,alerta`. Los datos se escriben recursivamente en el archivo plano `/data/historial.csv`, persistido de forma segura en el host a través de un bind mount del contenedor Docker.
2. **Historial de Accesos (SQLite)**: El backend FastAPI gestiona la base de datos SQLite `access_control.db`. Esta almacena en la tabla `usuarios` la identidad de las personas autorizadas junto con el blob serializado de su firma facial de 128 dimensiones. La tabla `historial` registra cada evento de la puerta principal (“Entrada Automática”, “Apertura Manual”, “Desconocido Detectado” o “Sin Rostro”) junto con su respectiva marca temporal en hora chilena.

8. Problemas Encontrados y Soluciones Aplicadas

Durante las pruebas de integración del sistema distribuido con hardware físico, surgieron tres problemas complejos que requirieron optimizaciones avanzadas de software.

8.1. Problema 1: Apagado Inexistente del LED de Alerta

Descripción: Originalmente, el LED y el buzzer de alerta se activaban de forma correcta al detectarse una variable crítica en Node-RED, pero permanecían encendidos de forma indefinida hasta que se enviaba un comando manual desde Telegram o el dashboard. Esto presentaba un riesgo de sobrecalentamiento de actuadores y contaminación acústica ante falsas alarmas temporales.

Solución: Se programó un contador de tiempo lógico en Node-RED utilizando variables de flujo (`flow.get("lastAlertTime")`). Ahora, cada vez que una lectura supera los límites, se refresca la marca temporal del evento. Si los sensores retornan a valores normales, el flujo ejecuta una verificación periódica. Tras transcurrir 15 minutos (900,000 ms) consecutivos sin lecturas críticas en el bus, Node-RED publica de forma automática comandos "OFF" a los actuadores para normalizar el hogar sin requerir intervención del usuario.

8.2. Problema 2: Inestabilidad y Pérdida en el Reconocimiento Facial

Descripción: La visión artificial experimentaba tres fallas severas: (a) El reconocimiento fallaba frecuentemente por cambios en las condiciones de luz natural en el acceso. (b) Las imágenes transmitidas por MQTT aparecían cortadas o con líneas grises debido al choque de paquetes de red al transmitir streams continuos pesados. (c) La tasa de falsos rechazos era muy alta debido a límites de tolerancia extremadamente rígidos en el algoritmo. Además, si el sensor MAX4466 enviaba múltiples picos de sonido seguidos, se disparaban ráfagas de fotos concurrentes sin pausar, saturando el broker y bloqueando la evaluación del backend.

Solución: Se realizaron las siguientes modificaciones correctoras en el backend de FastAPI y en el firmware:

1. Se eliminó el servidor HTTP de streaming local de la ESP32-CAM. Ahora la cámara trabaja de forma pasiva enviando únicamente ráfagas de fotos controladas por MQTT tras recibir el evento de captura.
2. Se redujo el tamaño del burst de frames en la ESP32-CAM: ahora captura y transmite ráfagas cortas de 3 a 5 imágenes espaciadas por 250ms en lugar del esquema anterior de 10 frames continuos de video. Esto mitigó el choque de paquetes de red y solucionó la corrupción de imágenes JPEG.
3. Se implementaron parámetros de comparación flexibles en el archivo del backend `processor.py`: la tolerancia geométrica se redujo a 0.35 (haciendo el emparejamiento de vectores de distancia dlib más selectivo y seguro) y la confianza mínima se fijó en 0.50. Para robustecer la decisión, el sistema requiere confirmación en al menos 3 frames consecutivos del burst para dar por válida la identidad del usuario.
4. En el backend FastAPI, se introdujo un semáforo asíncrono global (`_capture_lock = asyncio.Lock()`) y un flag de captura en el estado de la aplicación (`app.state.capturing`).

Si se detectan múltiples eventos acústicos seguidos, el backend ignora las peticiones adicionales mientras una sesión de captura de 10 segundos esté activa, evitando la superposición de tareas.

8.3. Problema 3: Caídas Frecuentes a Cero del Sensor de Gas MQ-2

Descripción: El sensor MQ-2 presentó un comportamiento físico anómalo: al exponerse a altas concentraciones de gas licuado o humo denso que superaban el rango máximo nominal del sensor, la lectura analizada en el MKR1000 se saturaba y caía abruptamente a 0 de forma momentánea antes de recuperarse. En el código original de control automático, esto provocaba que el sistema interpretara la caída a 0 como un retorno a la normalidad y apagara inmediatamente los actuadores de alerta, provocando un parpadeo intermitente peligroso en medio de una fuga real.

Solución: Este fallo físico se resolvió por software a través de dos estrategias combinadas en el flujo de Node-RED:

1. **Ajuste del Umbral Crítico:** Se configuró el umbral de alarma de gas (`gas_threshold`) en el valor máximo seguro de 1020 ppm (dentro de un rango de ADC de 10 bits de 0 a 1023). Al ubicar el umbral justo al borde de la saturación del sensor, se garantiza que la alarma se active inmediatamente antes de que ocurra la caída física del voltaje.
2. **Lógica de Latch y Evaluación Condicional:** En el nodo `rules_evaluator` de Node-RED, la lógica de alarma se evalúa de manera condicionada mediante la sentencia `if (gas && gas > gasThreshold)`. En Javascript, el valor 0 se evalúa como falso (*falsy*), por lo que si el sensor cae a 0, la regla de activación no se ejecuta de nuevo. Sin embargo, debido al enclavamiento (latch) por software implementado con la variable global de tiempo `flow.set("lastAlertTime", Date.now())`, el estado de alerta general permanece en "ON" y no apaga los actuadores. Node-RED requiere que pasen 15 minutos consecutivos con lecturas estrictamente válidas y por debajo del umbral para enviar la orden de apagado. De este modo, las caídas transitorias a 0 son ignoradas por el sistema físico de advertencia.

9. Conclusiones

La implementación de este prototipo domótico de SmartHome IoT demuestra el valor de las arquitecturas modulares y distribuidas para la automatización de viviendas. Mediante el uso de un bus de datos MQTT centralizado, se logró desacoplar las tareas de lectura física y actuación (MKR1000) de los procesos computacionalmente intensivos como el reconocimiento facial (FastAPI) y la orquestación del dashboard web junto con la lógica de control (Node-RED).

Entre las principales lecciones técnicas obtenidas se destaca la importancia de diseñar sistemas de tolerancia a fallos por software para mitigar las limitaciones físicas

inherentes del hardware económico. La inestabilidad en las lecturas del sensor de gas MQ-2 o la saturación de corriente por ráfagas concurrentes en la cámara demostraron que la programación basada puramente en estados instantáneos de sensores es insuficiente para entornos críticos de seguridad. La incorporación de técnicas de enclavamiento temporal (latch), filtros de concurrencia mediante cerrojos de exclusión mutua (**Locks**) y la calibración fina de umbrales permitieron transformar componentes inestables en una solución robusta y confiable.

Como trabajo futuro, se recomienda la transición del protocolo de red a conexiones con seguridad TLS y autenticación por usuario/contraseña en Mosquitto, la integración de un esquema de alimentación ininterrumpida (UPS) para prevenir caídas ante cortes de suministro eléctrico, y la implementación de técnicas de aprendizaje federado para entrenar el modelo de visión artificial localmente con mayor cantidad de imágenes de muestra ante condiciones adversas de iluminación.